

GestorCMB — Guía de estudio

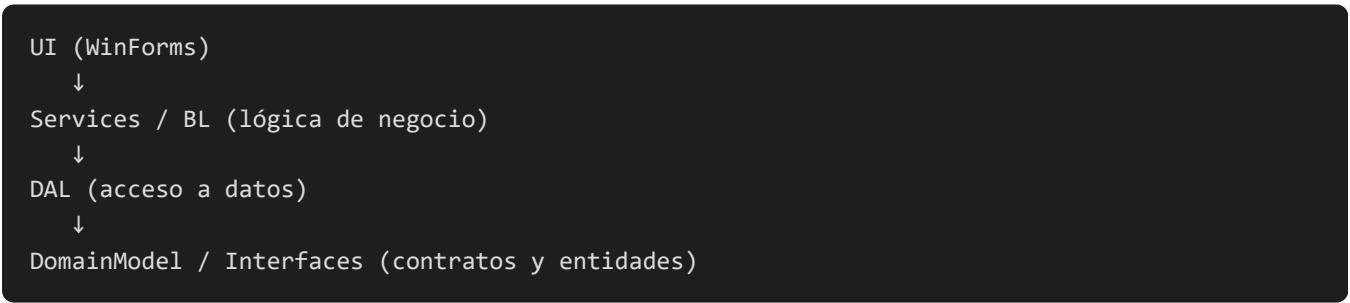
Documento de apoyo para la defensa del proyecto · CMB Instalaciones

Contenido

- 1. Arquitectura general y las 3 bases de datos
- 2. Patrones de diseño usados
- 3. El patrón Composite explicado a fondo
- 4. Familia y Patentes (RBAC): ¿está bien la lógica?
- 5. Multi-idioma (i18n)
- 6. Hasher y encriptación
- 7. Sistema de logs
- 8. Acceso a datos: Entity Framework vs. consultas (ADO.NET)
- 9. UI y gráficos del análisis
- 10. Mejoras de seguridad aplicadas
- 11. Posibles preguntas del tribunal

1. Arquitectura general y las 3 bases de datos

El sistema sigue una **arquitectura por capas** clásica, con dependencias en una sola dirección (de arriba hacia abajo). Cada capa tiene una responsabilidad concreta:



Una decisión de diseño importante es que el proyecto **no usa una sola base de datos, sino tres**, cada una con un propósito distinto y un mecanismo de acceso distinto:

Base de datos	Para qué sirve	Cómo se accede
GestorCMB	Negocio: proyectos, clientes, informes de compra, etc.	Entity Framework 6 (EDMX)
GestorCMB_Users	Seguridad: usuarios, roles (Familia), permisos (Acceso)	ADO.NET (consultas SQL a mano)
GestorCMB_Logs	Bitácora: registros de eventos del sistema	ADO.NET

Por qué separar. Separar seguridad y logs del negocio es una buena práctica: si alguien compromete la base de negocio, no obtiene de paso las credenciales ni puede borrar la bitácora. Además permite escalar o respaldar cada una por separado. Es un buen argumento para defender.

2. Patrones de diseño usados

Patrón	Dónde	Para qué
Composite	Permisos (IPermission / Acceso / RolCompuesto / Usuario)	Tratar permisos sueltos y grupos de permisos de forma uniforme
Repository	Capa DAL (UsuarioRepository, FamiliaRepository, etc.)	Aislar el acceso a datos detrás de una interfaz
Unit of Work	SqlUnitOfWork (EF) y SqlLoginUnitOfWork (ADO.NET)	Agrupar operaciones y manejar la conexión/transacción
Strategy + Composite	Logging (distintos destinos de log)	Elegir y combinar cómo se escribe el log (archivo, BD, ambos)
Singleton	LanguageService (idioma)	Un único punto de verdad para el idioma activo
Facade	LoggerLogic	Esconder la complejidad del logging tras una fachada simple

3. El patrón Composite explicado a fondo

El Composite es el patrón estrella del proyecto y casi seguro te van a preguntar por él. La idea del patrón es: **poder tratar a un objeto individual y a un grupo de objetos exactamente igual**, a través de una misma interfaz. El cliente no necesita saber si está hablando con una hoja o con una rama del árbol.

Los participantes en este proyecto

Rol en el patrón	Clase	Qué hace
Component (la interfaz común)	<code>IPermission</code>	Declara <code>bool TienePermiso(TipoPermiso) , Nombre , Id</code>
Leaf (la hoja)	<code>Acceso</code>	Un permiso individual. <code>TienePermiso</code> devuelve <code>DataKey == permiso</code>
Composite (la rama)	<code>RolCompuesto</code>	Contiene una lista de <code>IPermission</code> hijos y delega en ellos
Client	<code>Usuario</code>	Consulta permisos sin saber si son hojas o ramas

La hoja: Acceso

Un **Acceso** es un permiso atómico (ejemplo: "ver proyectos"). Su lógica es trivial: tiene un permiso si y solo si es ese permiso.

```
// DomainModel/Login/Acceso.cs (esencia)
public bool TienePermiso(TipoPermiso permiso)
{
    return DataKey == permiso;
}
```

La rama: RolCompuesto

Un **RolCompuesto** (un rol, ej. "Comprador") no tiene un permiso propio: tiene **hijos**, que pueden ser hojas (Acceso) u **otros roles**. Para responder "¿tenés tal permiso?" le pregunta recursivamente a cada hijo:

```
// Services/RoleService/RolCompuesto.cs (esencia)
private readonly List<IPermiso> _hijos = new List<IPermiso>();

public void AgregarHijo(IPermiso permiso) { _hijos.Add(permiso); }
public void QuitarHijo(IPermiso permiso) { _hijos.Remove(permiso); }

public bool TienePermiso(TipoPermiso permiso)
{
    // delega en los hijos: la recursión clave del Composite
    return _hijos.Any(h => h.TienePermiso(permiso));
}
```

El truco del patrón. Fijate que `RolCompuesto.TienePermiso` llama a `h.TienePermiso(...)` sin saber si `h` es un Acceso o un RolCompuesto. Si es hoja, compara directo; si es rama, vuelve a bajar. Esa indiferencia es *todo* el Composite. La recursión termina sola cuando llega a las hojas.

El cliente: Usuario

```
// DomainModel/Login/Usuario.cs
private readonly List<IPermiso> _permisos = new List<IPermiso>();

public bool TienePermiso(TipoPermiso permiso)
{
    return _permisos.Any(x => x.TienePermiso(permiso));
}
```

El usuario puede tener mezclados, en su lista de permisos, tanto accesos directos como roles completos, y los consulta a todos igual. Eso es exactamente lo que el patrón promete.

4. Familia y Patentes (RBAC): ¿está bien la lógica?

En la terminología del proyecto (según tu propio `Docs/Diagramas_Modelo.md`):

- **Familia = Rol** (ej. "Gerente", "Comprador")
- **Acceso = Patente** (un permiso individual)

El control de acceso es **RBAC con doble vía**, lo cual es correcto y flexible:

1. **Vía directa:** `Usuario_Acceso` — se le da un permiso suelto a un usuario.
2. **Vía por rol:** `Usuario_Familia` → `Familia_Acceso` — el usuario pertenece a un rol, y el rol tiene permisos.

Lo que está bien. La construcción del árbol Composite en `RolesService.ListarRoles()` es correcta. La protección del rol admin (no se puede borrar/modificar, lanza `AppException("err_admin_protegido")`) está bien pensada. El seed de roles predefinidos (Operario, Jefe de obra, Comprador, Gerente) en `EnsureRolesPredefinidosExist()` también.

El detalle a tener listo (te lo pueden marcar). Existe en la base de datos y en los diagramas una tabla `Familia_Familia` que serviría para anidar roles dentro de roles (la jerarquía recursiva que justifica plenamente el Composite). **Pero esa tabla no se usa en el código:** `FamiliaRepository` tiene métodos para `Familia_Acceso`, `Usuario_Familia` y `Usuario_Acceso`, pero *ningún* método para `Familia_Familia`.

Qué significa. En la práctica el árbol Composite hoy tiene profundidad 2 (Rol → Accesos), no rol-dentro-de-rol. El patrón está bien implementado y funciona, pero la jerarquía recursiva completa que muestran los diagramas es *aspiracional*, no está cableada en el código todavía.

Cómo defenderlo. No es un error: es una capacidad preparada en el diseño (la interfaz y el Composite ya lo soportan) que aún no se expuso por la UI/DAL. Si el tribunal pregunta, decí exactamente eso — demuestra que entendés tu propio código mejor que si lo escondieras.

5. Multi-idioma (i18n)

El multi-idioma combina tres piezas:

1. **LanguageService (Singleton):** mantiene el idioma activo en un único lugar.
2. **Resources.resx + ResourceManager de .NET:** los textos se guardan como pares clave→valor por idioma (es, en, ...).
3. **Método T():** dada una clave (ej. `"btn_guardar"`) devuelve el texto en el idioma activo.

Como los formularios de WinForms no son `Localizable` nativamente en este patrón, cada Form implementa un método `AplicarTraducciones()` que recorre sus controles y les asigna el texto traducido vía `T()`. Se llama al cargar el formulario y al cambiar de idioma.

```
// patrón típico en cada Form
private void AplicarTraducciones()
{
    this.Text      = LanguageService.T("titulo_ventana");
    btnGuardar.Text = LanguageService.T("btn_guardar");
    lblNombre.Text  = LanguageService.T("lbl_nombre");
    // ...
}
```

6. Hasher y encriptación

Contraseñas: BCrypt

Las contraseñas se guardan con **BCrypt** (work factor 12), que ya incluye el *salt* embebido en el propio hash. Hay además un *fallback* a SHA256 para contraseñas viejas (legado), de modo que usuarios antiguos puedan seguir entrando y migrar al iniciar sesión.

```
// Interfaces/LoginInterfaces/IPasswordHasher.cs
public interface IPasswordHasher
{
    string Hash(string plainText);
    bool  Verify(string hash, string plainText);
}
```

BCrypt es la decisión correcta. Es lento a propósito (resistente a fuerza bruta), trae salt incorporado y es un estándar de la industria. Buen punto a favor en la defensa.

OTP / recuperación de contraseña (2FA por mail)

La recuperación de contraseña genera un código de un solo uso (OTP) de 6 dígitos, lo envía por mail (SMTP) y lo valida con vencimiento de 15 minutos. Sobre este flujo se aplicaron dos mejoras de seguridad (ver sección 10).

7. Sistema de logs

El logging usa **Strategy + Composite** detrás de una fachada (`LoggerLogic`) con inicialización perezosa (`Lazy<T>`):

- **Strategy:** cada destino de log (archivo, base de datos) es una estrategia intercambiable.
- **Composite:** el valor `LoggerType="both"` en `App.config` combina varias estrategias para escribir en todos los destinos a la vez.
- **Facade:** el resto del sistema solo habla con `LoggerLogic` , sin conocer los detalles.

Convención de niveles (según el proyecto). Info para acciones normales del usuario, Warn para situaciones recuperables o inesperadas no graves, Error para excepciones. No se loguean datos sensibles (contraseñas, OTP, tokens).

8. Acceso a datos: Entity Framework vs. consultas (ADO.NET)

Esta es otra pregunta muy probable. El proyecto usa **dos mecanismos distintos a propósito**:

	Entity Framework 6	ADO.NET (query manual)
Dónde	Base <code>GestorCMB</code> (negocio)	Bases <code>GestorCMB_Users</code> y <code>GestorCMB_Logs</code>
Estilo	Database-First, modelo EDMX (<code>GestorCMBEntityModel.edmx</code>), <code>DbContext = GestorCMBEntities</code>	<code>SqlConnection</code> + <code>SqlCommand</code> con parámetros
Unit of Work	<code>SqlUnitOfWork</code>	<code>SqlLoginUnitOfWork</code>

Ejemplo real de ADO.NET con parámetros (de `FamiliaRepository`), nótese el uso de parámetros `@f` que **previene inyección SQL**:

```
using (var cn = new SqlConnection(_cs))
using (var cmd = new SqlCommand(
    @"SELECT a.idAcceso, a.nombre, a.dataKey
    FROM Familia_Acceso fa
    JOIN Acceso a ON a.idAcceso = fa.idAcceso
    WHERE fa.idFamilia = @f
    ORDER BY a.nombre", cn))
{
    cmd.Parameters.AddWithValue("@f", idFamilia); // parametrizado = seguro
    cn.Open();
    // ...
}
```

Cómo justificar usar los dos. EF acelera el desarrollo del negocio (CRUD, navegación entre entidades). Para seguridad y logs, ADO.NET da control fino sobre las consultas y evita acoplar esas bases críticas al modelo EDMX del negocio. Es una separación deliberada, no una inconsistencia.

9. UI y gráficos del análisis

Navegación

La UI **no** usa MDI clásico. En su lugar usa un patrón de **intercambio de UserControls**: hay un contenedor principal y se van cargando/quitando UserControls (pantallas) dentro de él. Esto da una sensación de aplicación de página única, más moderna que ventanas hijas.

Gráficos

Los gráficos del análisis usan `System.Windows.Forms.DataVisualization.Charting.Chart` (la librería de charts nativa de WinForms). Se alimentan con datos agregados del negocio (por ejemplo, totales por proyecto/periodo) y permiten mostrar el análisis visual que presentan en la demo.

10. Mejoras de seguridad aplicadas

Durante esta revisión se aplicaron **dos mejoras de bajo esfuerzo y alto valor** en `BL/LoginBL/OtpService.cs`:

Mejora 1 — OTP guardado hasheado (no en texto plano)

Antes el OTP se guardaba tal cual en la columna `Otp`. Si alguien leía la base, veía el código. Ahora se guarda con BCrypt y se valida con `Verify`:

```
// al generar
usuario.Otp = _hasher.Hash(otp); // se guarda hasheado

// al validar
return _hasher.Verify(usuario.Otp, otp.Trim());
```

Mejora 2 — OTP con generador criptográfico

Antes usaba `new Random()`, que es predecible (un atacante que conoce la semilla puede anticipar códigos). Ahora usa un RNG criptográfico:

```
private static string GenerarOtpSeguro()
{
    using (var rng = RandomNumberGenerator.Create())
    {
        var bytes = new byte[4];
        rng.GetBytes(bytes);
        var valor = (int)(BitConverter.ToUInt32(bytes, 0) % 1000000);
        return valor.ToString("D6");
    }
}
```

No rompe nada. La columna `Otp` ya es texto, guarda igual un hash que un número. El mail al usuario sigue mostrando el código de 6 dígitos legible, que es lo correcto. `CambiarContraseña` ya hasheada bien la nueva clave y limpiaba `Otp` / `OtpExpiry`.

Pendiente opcional (no afecta el funcionamiento). La contraseña de aplicación de Gmail está escrita directo en `App.config` y quedó en el historial de git. El mail funciona igual, pero conviene **rotar esa contraseña en Gmail** (2 minutos) porque cualquiera con acceso al repo la tiene. Idealmente moverla a una variable de entorno o config local fuera de git.

11. Posibles preguntas del tribunal

Pregunta	Respuesta corta
¿Qué patrón usaron para los permisos y por qué?	Composite, para tratar permisos sueltos y roles igual a través de <code>IPermiso</code> .
¿Cómo funciona la recursión del Composite?	<code>RolCompuesto</code> delega <code>TienePermiso</code> en sus hijos sin saber si son hojas o ramas; corta solo al llegar a las hojas (<code>Acceso</code>).
¿Por qué tres bases de datos?	Separar negocio, seguridad y logs por aislamiento, seguridad y mantenibilidad.
¿Por qué EF en una y ADO.NET en otras?	EF agiliza el CRUD de negocio; ADO.NET da control fino y desacopla seguridad/logs del EDMX.
¿Cómo evitan inyección SQL?	Consultas parametrizadas (<code>@parametro</code> con <code>AddWithValue</code>).
¿Cómo guardan las contraseñas?	BCrypt (work factor 12, salt embebido), con fallback SHA256 para legado.
¿La jerarquía de roles anidados está implementada?	El diseño y el Composite lo soportan; la tabla <code>Familia_Familia</code> existe pero aún no se cablea en el DAL. Es capacidad preparada, no error.
¿Cómo manejan el multi-idioma?	Singleton <code>LanguageService</code> + <code>.resx</code> + <code>T()</code> + <code>AplicarTraducciones()</code> por formulario.